

5.IMAGE FILTER USING PYTHON

AIM:

To perform image smoothing and sharpening using spatial filtering techniques in Python with OpenCV.

INTRODUCTION:

Spatial filtering is a technique used in image processing to modify or enhance an image based on the intensity values of neighbouring pixels. It involves applying a filter (kernel) over the image to perform operations such as smoothing and sharpening.

- Smoothing filters (low-pass filters) reduce high-frequency components, such as noise and edges, resulting in a blurred image. This is helpful for noise reduction.
- Sharpening filters (high-pass filters) emphasize edges and fine details by enhancing areas of rapid intensity change, thus improving the visibility of structures.
These operations are commonly implemented using libraries like OpenCV, NumPy, and Matplotlib in Python for visualization and processing.

Code to import and show image in python:

```
img=cv2.imread(r"C:\Users\BME\Pictures\DIP3E_Original_Images_CH04\hand.tif")
plt.imshow(img)
plt.show()
```

Output image:



CODE FOR 2D CONVOLUTION USING AVERAGING:

AVERAGING:

Averaging is a basic smoothing technique where each pixel is replaced with the average value of its neighbors within a defined kernel size. This reduces sharp intensity variations and helps blur the image.

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
img=cv2.imread(r"C:\Users\BME\Pictures\DIP3E_Original_Images_CH04\hand.tif")
plt.imshow(img)
plt.show()
k=np.array([[0,0,0],[0,1,0],[0,0,0]])
flt=cv2.filter2D(src=img,ddepth=-1,kernel=k)
plt.imshow(flt)
plt.show()

```

Output image:



Comparing images of Comparing Techniques(Averaging & Blur Box):

Blur Box:

Box blur is a type of averaging filter where all kernel values are equal. It normalizes the pixel values by dividing the sum of the kernel by its total number of elements. It's simple but may not preserve edges well.

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
img=cv2.imread(r"C:\Users\BME\Pictures\DIP3E_Original_Images_CH04\hand.tif")
plt.imshow(img)
plt.show()
k=np.array([[0.1111111,0.1111111,0.1111111],[0.1111111,0.1111111,0.1111111],[0.1111111,0.1111111,0.1111111]])
flt=cv2.filter2D(src=img,ddepth=-1,kernel=k)

k1=np.array([[0,0,0],[0,1,0],[0,0,0]])
flt1=cv2.filter2D(src=img,ddepth=-1,kernel=k1)

plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(flt)

```

```
plt.title('Blur Box')
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(flt1)
plt.title('Averaged Image')
plt.axis('off')

plt.subplot(1,3,3)
plt.imshow(img)
plt.title('Original Image')
plt.axis('off')
plt.show()
```

Output image:



(Comparison of Original Image with Two Smoothen Image obtained using different technique)

GAUSSIAN BLUR:

Gaussian blur applies a weighted average, giving more importance to pixels closer to the center. It uses a Gaussian function to assign weights, making it smoother and better at preserving edges than box blur. It's commonly used for noise reduction

Python code for Gaussian Blur:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img=cv2.imread(r"C:\Users\BME\Pictures\DIP3E_Original_Images_CH04\hand.tif")
flt=cv2.GaussianBlur(img,(5,5),0)
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img)
plt.title('Original Image')
plt.axis('off')

plt.subplot(1,2,2)
```

```
plt.imshow(flt)
plt.title('Gaussian Smoothed Image')
plt.axis('off')

plt.show()
```

Output image:



(Comparison of Original Image with Gaussian Smoothed Image)

Code To remove Salt and Pepper Noise:

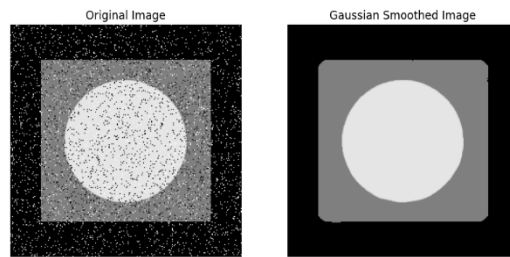
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img=cv2.imread(r"C:\Users\BME\Pictures\DIP3E_Original_Images_CH05\sp
.tif")
flt=cv2.medianBlur(img,15)
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img)
plt.title('Original Image')
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(flt)
plt.title('Gaussian Smoothed Image')
plt.axis('off')

plt.show()
```

Output image:



(a-Image with Salt & Pepper Noise b-Noise removed Image)

SHARPENING:

Sharpening enhances edges and fine details in an image by increasing contrast around edges. It uses high-pass filters to highlight rapid intensity changes, making the image appear clearer and more defined. Common methods include Laplacian and Unsharp Masking.

Code to sharpen the image:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img=cv2.imread(r"C:\Users\ashle\OneDrive\Pictures\skull.tif")
plt.imshow(img)
plt.show()
k=np.array([0,-1,0],[-1,5,-1],[0,-1,0])
flt=cv2.filter2D(src=img,ddepth=-1,kernel=k)
plt.imshow(flt)
plt.show()
```

Input image:



(Unsharpen Image)

Output image:



(Sharpened image)

Code for Sharpening Image using Low pass operation:

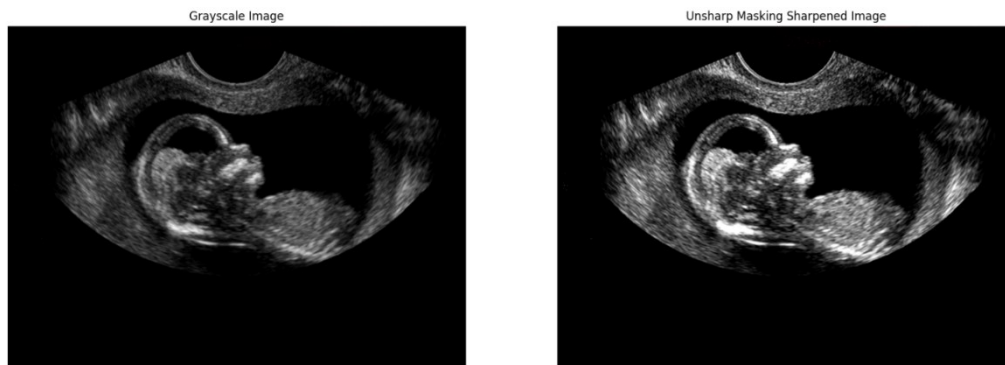
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```

img=cv2.imread(r"C:\Users\BME\Pictures\DIP3E_Original_Images_CH02\DI
P3E_Original_Images_CH01\fetus.tif")
blur=cv2.GaussianBlur(img,(5,5),0)
sharp=cv2.addWeighted(img,2.5,blur,-1,0)
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(img,cmap='gray')
plt.title('Grayscale Image')
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(sharp,cmap='gray')
plt.title('Unsharp Masking Sharpened Image')
plt.axis('off')
plt.show()

```

Output image:



(Unsharpen Image vs Sharpen Image)

Code for High-Pass Filtering for Edge-Based Image Sharpening (Boundary Enhancement):

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
img=cv2.imread(r"C:\Users\BME\Pictures\DIP3E_Original_Images_CH02\DI
P3E_Original_Images_CH01\skull.tif")
blur=cv2.GaussianBlur(img,(5,5),0)
hp=cv2.subtract(img,blur)
hp=cv2.normalize(hp,None,0,255,cv2.NORM_MINMAX)
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img,cmap='gray')
plt.title('Grayscale Image')
plt.axis('off')

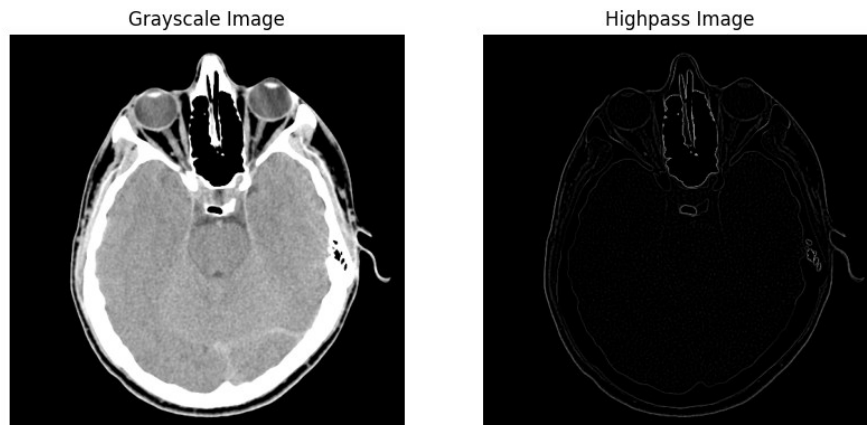
plt.subplot(1,2,2)
plt.imshow(hp,cmap='gray')

```

```
plt.title('Highpass Image')
plt.axis('off')

plt.show()
```

Output image:



(Unsharpen image vs Boundary Enhanced Image)

LAPLACIAN SHARPENING:

Laplacian sharpening is an edge enhancement technique that uses the Laplacian operator, a second-order derivative filter, to highlight regions of rapid intensity change in an image (i.e., edges).

- The Laplacian calculates the sum of second-order derivatives in both horizontal and vertical directions.
- It detects edges by identifying areas where pixel values change sharply.
- The result of the Laplacian is usually added back to the original image to sharpen it, making the edges more prominent.

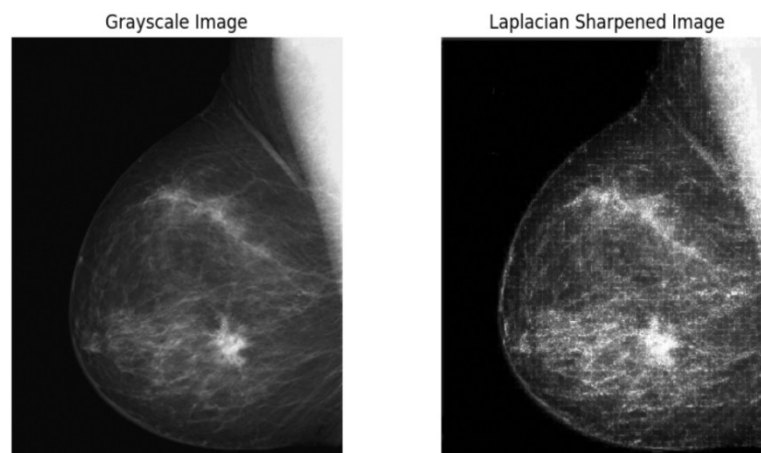
This method is sensitive to noise, so it's often applied after smoothing (e.g., Gaussian blur) to get cleaner edge detection.

Code to enhance using Laplacian sharpening:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread(r'C:\Users\BME\Pictures\IP
folder\IPLAB_2025\DIP3E_Original_Images_CH03\Fig0304(a)(breast_digital_Xray).tif')
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
laplacian = cv2.Laplacian(gray_img, cv2.CV_64F, ksize=3)
laplacian = cv2.convertScaleAbs(laplacian)
sharpened_image = cv2.add(gray_img, laplacian)
plt.figure(figsize=(10,5))
```

```
plt.subplot(1,2,1)
plt.imshow(gray_img, cmap='gray')
plt.title('Grayscale Image')
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(sharpened_image, cmap='gray')
plt.title('Laplacian Sharpened Image')
plt.axis('off')
plt.show()
```

Output image:



(Original Image vs Laplacian Sharpened Image)

INFERENCE :

In biomedical imaging, spatial filtering techniques play a vital role in enhancing image quality for accurate diagnosis and analysis.

- Smoothing techniques like Gaussian Blur are highly effective in removing noise from X-rays, MRI, and ultrasound images, which helps radiologists focus on relevant structures without distraction from artifacts.
- Sharpening techniques, such as Laplacian filtering and unsharp masking, enhance edges and fine details, making it easier to detect tumor boundaries, bone fractures, blood vessel outlines, and other critical anatomical features.

Thus, these filtering operations support improved image interpretation, segmentation, and feature extraction, which are essential steps in biomedical diagnosis and computer-aided detection (CAD) systems.

CONCLUSION :

This experiment successfully demonstrated how spatial filtering techniques can be applied to biomedical images to improve their diagnostic value.

- Smoothing filters help in noise suppression for clearer medical images.
- Sharpening filters enhance critical features and boundaries, aiding in better visualization and analysis of anatomical structures.

These methods are fundamental in biomedical image preprocessing, directly impacting the performance of diagnostic tools, deep learning models, and clinical decision support systems.